



Week 2 - Fine-tuning and evaluating large language models

Learn about

- [Instruction Fine-Tuning](#)
- [Parameter efficient fine-tuning \(PEFT\)](#)

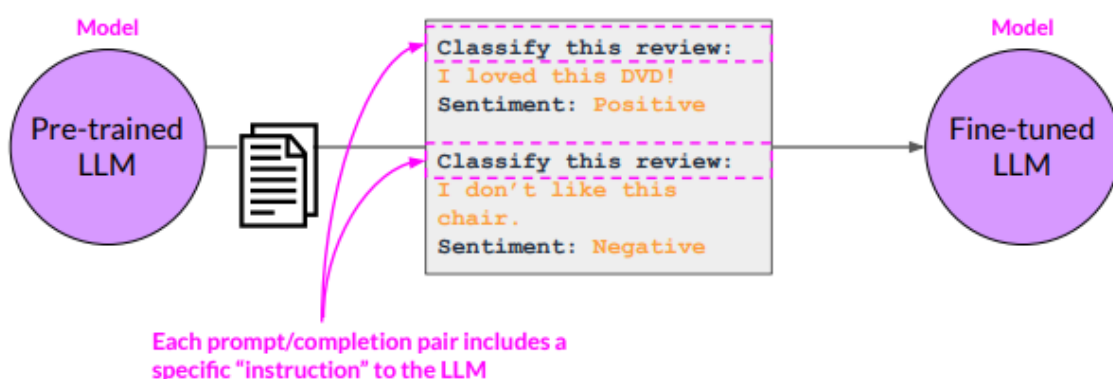
Instruction Fine-Tuning

In-context learning with One-shot and few-shot inferences help model identify and learn instructions as part of prompt, but for smaller models it doesn't always work, and take up valuable space in context window reducing the amount of other useful information you want to include.

Fine Tuning is a Supervised Learning process where you use a dataset of labelled examples to update the weights of the LLM. The labelled examples are prompt - completion pairs and leads to performance improvement. Fine tuning in general means Instruction fine-tuning.

- **Instruction fine tuning** where all of the model weights are updated is called full fine-tuning leading to new version of the model with updated weights. Note that the process requires storing all the model weights, gradients and optimizer states in the memory as pre-training

LLM fine-tuning

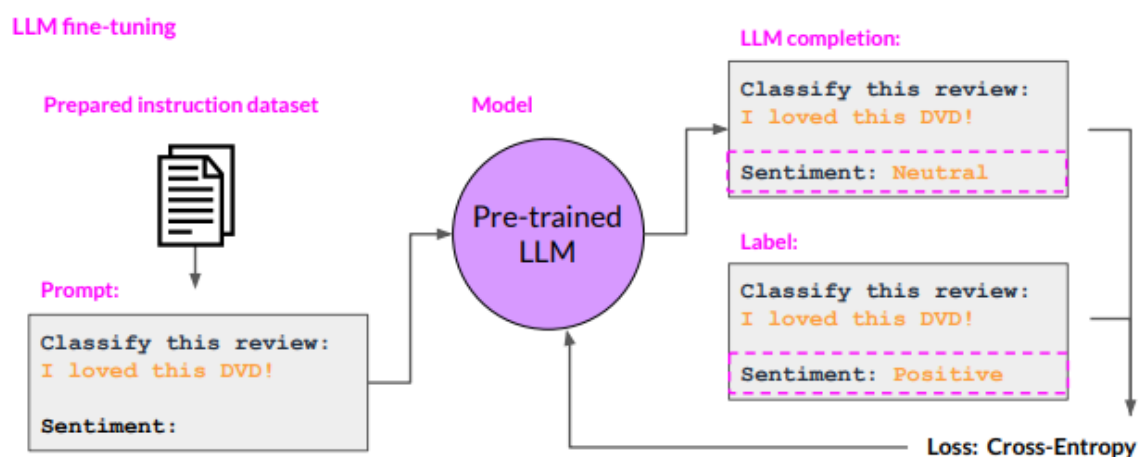


- Step 1: Preparing your training data

Developers have assembled prompt templates to take advantage of existing datasets like Amazon product reviews and turn them into instruction prompt datasets for fine-tuning for classification, text-generation and text-summarization tasks.

- Step 2: Divide datasets into training, validation and test split

Fine-tune using the examples in the training dataset and compare the outputs of the completion with the prediction completion using **Cross Entropy Loss Function**



Fine-tuning on a single task

In most cases the pre-trained LLM may only be required to perform on a single task. So 500-1000 good examples can lead to good performance

Catastrophic forgetting

While the model performs well on the single task for which it was fine-tuned but the performance degrades on other tasks is called catastrophic forgetting.

- Check whether your use-case is impacted by catastrophic forgetting, and it may not be
- Other way to get around is fine-tune on multiple tasks at the same time - using 50k-100K new examples and thus more compute and memory
- Consider Parameter efficient fine-tuning (PEFT). The model preserves the weight of the original LLM while training only on small number of task-

specific adapter layers and parameters. More robust to catastrophic forgetting

Multi-task instruction fine-tuning

Finetuning with examples related to multiple tasks

- Drawback is you need 50K - 100K examples in training set. However it can be really worth the effort to assemble this dataset

Fine-tuned LAnguage Net (FLAN),

- Authors called it the metaphorical dessert to the main course of pretraining
- Last step - Specific set of instructions used to fine-tune different models
- FLAN-T5: is the Instruct version of T5 foundation model
- FLAN-PALM: Is the instruct version of PALM Foundation model
- FLAN-T5 is a great general purpose instruct model. In total, it's been fine tuned on 473 datasets across 146 task categories

SAMsum: A dataset for conversation

- Crafted for specific purpose of creating a high quality dataset for language model has around 10K conversations with summary
- These conversation-summary pairs are passed through a prompt template that generates prompt in many different ways of asking the same thing so that the model generalizes better

```

"samsun": [
  ("{{dialogue}}\nBriefly summarize that dialogue.", "{{summary}}"),
  ("Here is a dialogue:\n{{dialogue}}\n\nWrite a short summary!",
   "{{summary}}"),
  ("Dialogue:\n{{dialogue}}\n\nWhat is a summary of this dialogue?",
   "{{summary}}"),
  ("{{dialogue}}\n\nWhat was that dialogue about, in two sentences or less?",
   "{{summary}}"),
  ("Here is a dialogue:\n{{dialogue}}\n\nWhat were they talking about?",
   "{{summary}}"),
  ("Dialogue:\n{{dialogue}}\n\nWhat were the main points in that "
   "conversation?", "{{summary}}"),
  ("Dialogue:\n{{dialogue}}\n\nWhat was going on in that conversation?",
   "{{summary}}"),
]

```

Model evaluation

Accuracy used for traditional machine learning models fail here because the output of LLMs is not deterministic.

ROUGE (Recall Oriented Understudy for Gesting Evaluation)

- Used for text summarization tasks
- Compares summaries to one or more summaries

Focused on individual words

- ROUGE-1 Recall = Unigram matches / Unigrams in references: Recall of unigrams in generated text to those in the reference text
- ROUGE-1 Precision = Unigram matches / Unigrams in output: Precision of unigrams in generated text to those in the output text
- ROUGE-1 F1 Score= Harmonic mean between precision and recall = $2 \frac{(\text{precision} \times \text{recall})}{(\text{precision} + \text{recall})}$

The Scores would remain the same if there is "not" in the sentence. The performance can be better measured with bigrams. ROUGE - 2. Scores are lower then ROUGE-1.

You can go to 3,4 and so on. Instead look for **longest common subsequence (LCS)** in generated output and reference text. It is called **ROUGE-L**

Reference (human):

It is cold outside.

Generated output:

It is very cold outside.

$$\text{ROUGE-L Recall:} = \frac{\text{LCS}(\text{Gen}, \text{Ref})}{\text{unigrams in reference}} = \frac{2}{4} = 0.5$$

$$\text{ROUGE-L Precision:} = \frac{\text{LCS}(\text{Gen}, \text{Ref})}{\text{unigrams in output}} = \frac{2}{5} = 0.4$$

$$\text{ROUGE-L F1:} = 2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = 2 \frac{0.2}{0.9} = 0.44$$

Note: Only compare ROUGE Scores for the same task. ROUGE Scores for different tasks are not comparable to one another.

ROUGE Hacking

If words are just repeated, the score could be perfect. To avoid this, make sure to clip the instance to maximum count of word in the reference text.

Reference (human):

It is cold outside.

Generated output:

cold cold cold cold

$$\text{ROUGE-1 Precision} = \frac{\text{unigram matches}}{\text{unigrams in output}} = \frac{4}{4} = 1.0$$

$$\text{Modified precision} = \frac{\text{clip}(\text{unigram matches})}{\text{unigrams in output}} = \frac{1}{4} = 0.25$$

Even the ordering of words is different the ROUGE-1 score would be perfect. This can be avoided with different n-gram sizes, which would be dependent on the use-case and the sentence size.

Generated output:

outside cold it is

$$\text{Modified precision} = \frac{\text{clip}(\text{unigram matches})}{\text{unigrams in output}} = \frac{4}{4} = 1.0$$

BLEU (Bilingual Evaluation Understudy).

- Used to evaluate machine translation tasks by comparing to human generated translations
- average precision over multiple n-grams

BLEU metric = Avg(precision across range of n-gram sizes)

Reference (human):

I am very happy to say that I am drinking a warm cup of tea.

Generated output:

I am very happy that I am drinking a cup of tea. - BLEU 0.495

I am very happy that I am drinking a warm cup of tea. - BLEU 0.730

I am very happy to say that I am drinking a warm tea. - BLEU 0.798

I am very happy to say that I am drinking a warm cup of tea. - BLEU 1.000

Benchmarks

Using pre-existing datasets to evaluate model performance

GLUE

General Language Understanding Evaluation introduced in 2018. Collection of Natural Language tasks - question answering and sentiment analysis etc.

SUPERGLUE

Introduced in 2019, to overcome the limitations in GLUE with additional tasks and more challenging versions of previous tasks like multi-sentence reasoning and reading comprehension

Some large language models can match human level performance on these benchmarks, but may be underperforming on other series of tasks. Here are some other benchmarks

MMLU (Massive Multitask Language Understanding)

Designed specifically for modern LLMs and are tested on elementary mathematics, US history, computer science, law and more

Big-bench

204 tasks. Also leads to large inference cost for running these benchmarks

HELM (Holistic Evaluation of Language Models)

- Aims to improve the transparency of models, and guide which models perform better on which tasks
- Takes multi-metric approach measuring 7 metrics across 16 core scenarios ensuring tradeoffs between models and metrics are clearly exposed.

- Includes metrics of fairness, bias and toxicity

Parameter efficient fine-tuning (PEFT)

- Less prone to catastrophic forgetting due to Full Fine-tuning
- Often can be conducted on a single GPU
- Most of the layers are Frozen for training. Only 15-20% of the size of the model actually trained
 - Either trained with additional layers
 - Freezing most of the layers and training only last few layers
- Full Fine-tuning leads to multiple version of the same size model, can create storage problems. With PEFT, training only small number of weights leads to smaller footprint
- PEFT trained for each task can be swapped in or out depending on the task.

Trade-offs

- Parameter efficiency
- Memory efficiency
- Training speed
- Model performance
- Inference Costs

Three main classes of PEFT methods

- **Selective:** Fine-tune only a subset of the original LLM parameters
 - select only individual parameter types, or selective layers
 - The performance is mixed and have significant tradeoffs between parameter efficiency and compute efficiency
- **Reparameterization:** Works with the original parameters of the model, but reduces the parameters to use Low-rank transformations of the original network weights. Example LoRA (Low-rank Adaptation).
- **Additive:** Freeze all existing layers for LLM and add trainable layers or parameters to the model. Also contains two methods

- **Adapters:** Add new trainable layers to the architecture of the model, either to encoder or decoder after the attention or feed forward layers
- **Soft prompt methods:** Keep the architecture fixed and frozen and focus on manipulating the input to achieve better performance
 - Adding trainable parameters to prompt embeddings, or
 - Keeping the input fixed and retraining the embedding weights

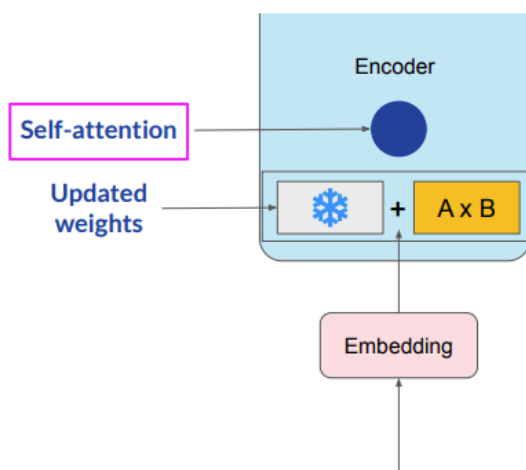
More on Prompt Tuning later

PEFT Techniques 1: LoRA (Low-rank Adaptation)

is a Parameter efficient fine-tuning (PEFT) technique that falls into **reparameterization** category.

How it works?

- Both encoder and decoder layers have Self Attention and feed forward layers whose weights are learned during pre-training
- Weights applied to the embedding vectors to calculate self-attention scores are fixed. Instead, 2 rank decomposition matrices are injected such the dimensions as a result of product of these matrices is the same as the weights they are modifying
- These smaller matrices are trained during fine-tuning keeping the original weights fixed. The weights trained are added to the original weights and the sum is replaced with the original



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**
3. Train the weights of the smaller matrices

Steps to update model for inference:

1. Matrix multiply the low rank matrices

$$\overline{B} * \begin{vmatrix} A \end{vmatrix} = \begin{vmatrix} A \times B \end{vmatrix}$$

2. Add to original weights

$$\begin{vmatrix} \text{Snowflake} \end{vmatrix} + \begin{vmatrix} A \times B \end{vmatrix}$$

- As the number parameters almost similar to the original model, little to no impact on the latency
- Researchers have found that the applying LoRA (Low-rank Adaptation) to only Self Attention layers can help achieve performance gains. Since most of the parameters are in self-attention layer.

Practical Example

Original transformer model have dimensions = $512 \times 64 = 32,768$ trainable parameters. Using the low rank matrix of size 8, you will have $8 \times 64 + 512 \times 8 = 4608$ trainable parameters (86% lower). Thus Parameter efficient fine-tuning (PEFT) can be done using a single GPU

How to chose rank?

Choosing rank is still an active area of research. Rank more than 16 resulted in plateau of the loss value. So a range between 4-32 can provide with a good trade-off between reducing trainable parameters and preserving performance.

Rank r	val_loss	BLEU	NIST	METEOR	ROUGE L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

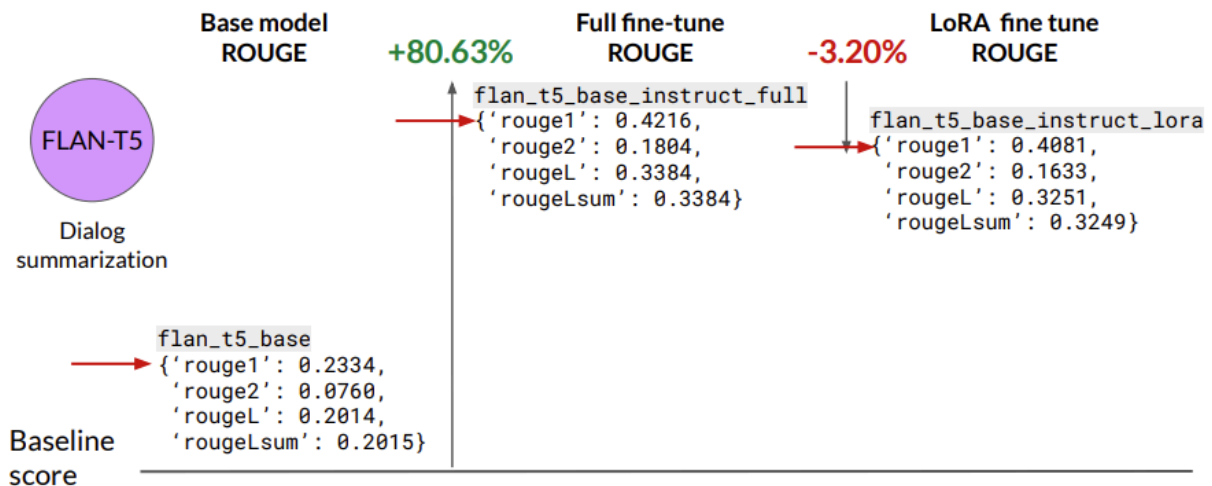
- Effectiveness of higher rank appears to plateau
- Relationship between rank and dataset size needs more empirical data

Source: Hu et al. 2021, "LoRA: Low-Rank Adaptation of Large Language Models"

How good are these models?

The PEFT tuned models perform at par with the full tuned models

Sample ROUGE metrics for full vs. LoRA fine-tuning



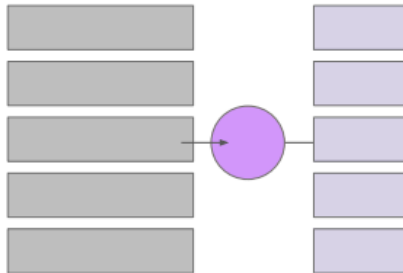
PEFT techniques 2: Soft prompts

Prompt Tuning is different from Prompt Engineering

- With Prompt Engineering, you work on the language of the prompt to get the completion that you want - trying different words/phrases to including examples. The goal is to help the model understand the nature of the task being carried out to generate a better completion
 - Limited by the amount of manual effort required to craft these prompts
 - Limited by the context-window of the model
 - Even after this you may not achieve the desired performance on the task
- With Prompt Tuning you add additional trainable tokens to your prompt and use supervised learning to determine the optimal values.
 - The set of trainable tokens is called the **soft prompt**.
 - Prepended to the embedding vectors to represent the input text
 - Same length as the embedding vectors
 - 20-100 tokens good for performance
 - Can represent any value between multidimensional space instead of just words or the tokens in the vocabulary of the LLM. However, they form tight semantic clusters meaning the words closest to the soft-prompt tokens have similar meaning

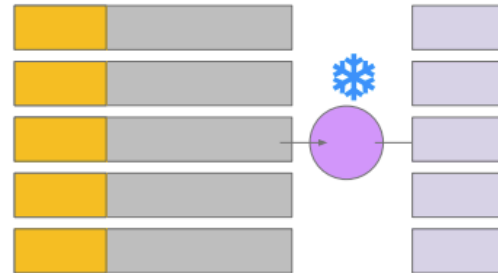
Fine Tuning vs Prompt Tuning

Weights of model updated during training



Millions to Billions of parameter updated

Weights of model frozen and soft prompt trained

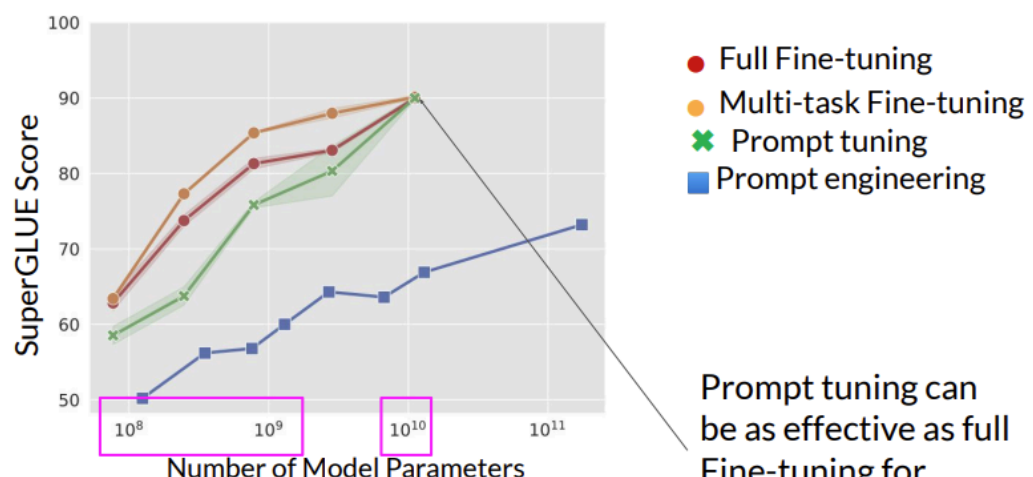


10K - 100K of parameters updated

- Weights of the model updated during fine-tuning, however weights of the model are frozen and instead the embedding vectors are updated overtime to optimize the completion of the prompt
- Similar to LoRA (Low-rank Adaptation), you can swap out different soft-prompts for the task at hand at inference time

Performance with Prompt Tuning

- Can be as effective as full Fine Tuning with larger models and offers significant boost in performance over Prompt Engineering alone



Source: Lester et al. 2021, "The Power of Scale for Parameter-Efficient Prompt Tuning"

Week 2 Resources

Below you'll find links to the research papers discussed in this weeks videos. You don't need to understand all the technical details discussed in these papers - **you have already seen the most important points you'll need to answer the quizzes** in the lecture videos.

However, if you'd like to take a closer look at the original research, you can read the papers and articles via the links below.

Generative AI Lifecycle

- **Generative AI on AWS: Building Context-Aware, Multimodal Reasoning Applications** - This O'Reilly book dives deep into all phases of the generative AI lifecycle including model selection, fine-tuning, adapting, evaluation, deployment, and runtime optimizations.

Multi-task, instruction fine-tuning

- **Scaling Instruction-Finetuned Language Models** - Scaling fine-tuning with a focus on task, model size and chain-of-thought data.
- **Introducing FLAN: More generalizable Language Models with Instruction Fine-Tuning** - This blog (and article) explores instruction fine-tuning, which aims to make language models better at performing NLP tasks with zero-shot inference.

Model Evaluation Metrics

- **HELM - Holistic Evaluation of Language Models** - HELM is a living benchmark to evaluate Language Models more transparently.
- **General Language Understanding Evaluation (GLUE) benchmark** - This paper introduces GLUE, a benchmark for evaluating models on diverse natural language understanding (NLU) tasks and emphasizing the importance of improved general NLU systems.
- **SuperGLUE** - This paper introduces SuperGLUE, a benchmark designed to evaluate the performance of various NLP models on a range of challenging language understanding tasks.
- **ROUGE: A Package for Automatic Evaluation of Summaries** - This paper introduces and evaluates four different measures (ROUGE-N, ROUGE-L, ROUGE-W, and ROUGE-S) in the ROUGE summarization evaluation

package, which assess the quality of summaries by comparing them to ideal human-generated summaries.

- **Measuring Massive Multitask Language Understanding (MMLU)** - This paper presents a new test to measure multitask accuracy in text models, highlighting the need for substantial improvements in achieving expert-level accuracy and addressing lopsided performance and low accuracy on socially important subjects.
- **BigBench-Hard - Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models** - The paper introduces BIG-bench, a benchmark for evaluating language models on challenging tasks, providing insights on scale, calibration, and social bias.

Parameter- efficient fine tuning (PEFT)

- **Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning** - This paper provides a systematic overview of Parameter-Efficient Fine-tuning (PEFT) Methods in all three categories discussed in the lecture videos.
- **On the Effectiveness of Parameter-Efficient Fine-Tuning** - The paper analyzes sparse fine-tuning methods for pre-trained models in NLP.

LoRA

- **LoRA Low-Rank Adaptation of Large Language Models** This paper proposes a parameter-efficient fine-tuning method that makes use of low-rank decomposition matrices to reduce the number of trainable parameters needed for fine-tuning language models.
- **QLoRA: Efficient Finetuning of Quantized LLMs** - This paper introduces an efficient method for fine-tuning large language models on a single GPU, based on quantization, achieving impressive results on benchmark tests.

Prompt tuning with soft prompts

- **The Power of Scale for Parameter-Efficient Prompt Tuning** - The paper explores "prompt tuning," a method for conditioning language models with learned soft prompts, achieving competitive performance compared to full fine-tuning and enabling model reuse for many tasks.